



SILESIAIAN UNIVERSITY OF TECHNOLOGY
FACULTY OF AUTOMATIC CONTROL, ELECTRONICS
AND COMPUTER SCIENCE

Engineer thesis

Distributed System for Image Processing

author: Mateusz Urbanek

supervisor: Przemysław Skurowski, PhD

Gliwice, April 2025

Oświadczenie

Wyrażam zgodę / ~~Nie wyrażam zgody~~* na udostępnienie mojej pracy dyplomowej / ~~rozprawy doktorskiej~~*.

Gliwice, dnia 30 kwietnia 2025

.....
(podpis)

.....
(poświadczenie wiarygodności
podpisu przez Dziekanat)

* podkreślić właściwe

Contents

1	Introduction	1
1.1	Problem statement	1
1.2	Document structure	2
2	State of the Art	3
2.1	RIDE project	3
2.2	Distributed Image Processing On A Network Of Workstations . . .	4
2.3	BOINC project	4
2.4	Summary	4
3	Problem analysis	5
3.1	Main issue	5
3.2	Functional requirements	5
3.3	Nonfunctional requirements	6
3.4	Use cases	6
3.5	Tools	7
3.5.1	Visual Studio Code	7
3.5.2	Windows Subsystem for Linux	8
3.5.3	protobuf-compiler	8
3.5.4	GNU Make	8
3.5.5	Docker	9
3.5.6	Portainer	9
3.5.7	pgAdmin 4	9
3.6	Technologies used in the project	9
3.6.1	Go	9

3.6.2	PostgreSQL	10
3.6.3	Shell script	10
3.6.4	gRPC framework	10
4	Project	11
4.1	Client-server communication	11
4.2	Architectural and design patterns	11
4.2.1	Sharded Services Design Pattern	12
4.2.2	HeartBeat Design Pattern	13
4.3	Remote Procedure Calls	13
4.3.1	Job RPC	13
4.3.2	Caching mechanism	15
4.3.3	Account management RPC	15
4.4	SQL Database system	17
5	Implementation	19
5.1	System specification	19
5.1.1	Manager Node	19
5.1.2	Worker Node	20
5.1.3	Client	22
5.1.4	Database	23
5.2	Manager Node	23
5.3	Worker Node	23
5.3.1	HeartBeat	24
5.3.2	Image Processing	24
5.4	Client	25
5.5	Image processing subprograms	26
5.6	Deployment process	27
5.6.1	Docker and docker-compose	27
5.6.2	Bare-metal deployment	29
5.6.3	Configuration Files Examples	31
5.6.4	CentOS and Fedora troubleshooting	32
5.6.5	Database deployment	34

6	Verification and validation	35
6.1	Methodology of testing	35
6.2	Testing results	36
6.2.1	Results - 80 small files	36
6.2.2	Results - 4 large files and Docker	37
6.2.3	Results - 4 large files and baremetal	37
6.3	Results analysis	38
6.3.1	Performance increase - 80 small files	38
6.3.2	Performance decrease - 4 large files and Docker	40
6.3.3	Performance increase - 4 large files and baremetal	42
6.3.4	Speedup analysis using Amdahl's law	44
7	Conclusions	47
7.1	Final project's outcome	47
7.2	Possible future development	47
7.3	Encountered difficulties and problems	48

Chapter 1

Introduction

1.1 Problem statement

Cloud, distributed and grid computing gains more and more popularity in recent years. We can observe the revenue growth trend among all cloud providers. AWS sales grew at a 33% clip in the first quarter of 2020, and Microsoft Azure revenue grew from \$50 billion in second quarter up to \$53.2 billion in third quarter [4]. Seeing the rise of giants like Amazon and Google, we have to agree, that this field of computer science is rapidly evolving, and will continue to evolve in the future.

The idea of distributed computing is not new - in fact, the first, simple implementations of distributed applications dates back to the early '70s. An example of one of the first distributed applications was e-mail [1]. Additionally, the first conferences concerning the topic of distributed systems, PODC and DISC, were held in 1982 and 1985 correspondingly.

With the amount of data generated every second reaching 1.7MB by internet users [2], and 350 million photos are uploaded just to Facebook [2], companies need some way to process the big data. Additionally, the system should be able to run on multiple processor architectures and operating systems - some companies are running x86-64 (also known as AMD64, x64, x86_64 or Intel 64), but at the same time, others can be running e.g. Amazon EC2 instances for their applications which are ARM64 based [9]. Also, there are multiple other architectures used in

the cloud, like IBM z/Architecture or Power ISA.

The main goal of this project is to create a simple yet effective distributed application that can perform distributed image processing on a large number of files, that is easy to scale and access. The effects of running this system were measured and compared with processing the same number of images on a single workstation.

The software was developed using the Go programming language. The gRPC framework was used in the implementation of communication between client and server and between servers internally. Additionally, pgx database driver and toolkit was used to provide a simple and concise way of communication of servers with the database. The computational part was written using the GoCV wrapper of the OpenCV library.

1.2 Document structure

This document is divided into 7 main chapters. The first chapter's purpose is to formulate the problem. It also describes the document structure. In the second chapter, the state of the art is described. It covers the literature research and describes existing solutions. The third chapter covers the problem analysis. It contains a description of both functional and nonfunctional requirements, use cases, tools and methodology of design and implementation. The fourth chapter is dedicated to the description of the project. It covers design patterns, client-server communication protocol and database scheme. During the fifth chapter, it is given the detail of implementation of the project. It describes system requirements, more interesting parts of code and it guides through deployment process. In the sixth chapter, the methodology of testing is described. This chapter also includes a results analysis. In the seventh chapter conclusions from the project are drawn. Here the outcome analysis is performed as well as possible future development.

Chapter 2

State of the Art

During the research phase, I found few articles that touched on the problem of the large-scale, batch, and real-time image processing on a distributed system. In this chapter, I want to explore and describe the most interesting existing solutions.

2.1 RIDE project

RIDE: real-time massive image processing platform on distributed environment [16] inspired me to focus on the communication scheme and scalability aspects of the system.

RIDE focuses on processing large-scale streams of images in real-time. The examples of such streams are live drone footages, image streams received from satellites and CCTV. The application itself consists of four main layers. In the application layer, the user creates an application for massive image processing and submits it to the master layer. In the master layer, the master node allocates broker and worker nodes in buffer and worker layers respectively and then distributes stream images onto partitions in broker nodes. Finally, it divides the application into tasks, and assigns them to worker nodes each processing one partition in buffer layer [16].

2.2 Distributed Image Processing On A Network Of Workstations

Distributed Image Processing On A Network Of Workstations [12] is a publication from National University of Singapore. It focuses This work was the first of its kind to realize a practical implementation of a load distribution strategy on a NOWs. As an application domain, they identified the image-processing domain as a natural candidate to implement the strategy proposed by DLT. The work states that image partitioning, and computing on a network of computers is a natural solution to low-level pixel-wise processing of an image, to minimize the processing time.

2.3 BOINC project

BOINC - Berkeley Open Infrastructure for Network Computing [14] - is an open-source software platform for computing using volunteered resources. It is middleware system for grid computing. Developed to support SETI@home project, but now it is used by multiple grid computing and volunteer computing projects like Rosetta@home and Einstein@home.

2.4 Summary

Software developed by me differs from the applications presented in previous sections, by taking data dependency into account. That meant that image division into multiple pieces is not done in this application. That, unfortunately, means that a single job can take much more time for more complex images. At the same time, simpler images are being processed much faster due to omitting the network overhead and not wasting time for splitting the image.

Chapter 3

Problem analysis

3.1 Main issue

Processing large amount of data on single workstation might take large amount of time. Adding more compute power to single workstation is not always possible. This creates opportunity for parallelization of the work between multiple workstations.

The application's goal is to create a simple, coherent API, that allows distributing work between network of workstations. The API must be flexible and not bound to single type of data.

3.2 Functional requirements

In software engineering a functional requirement defines what a system should do. It describes inputs, behavior and outputs that the system should provide. Before the start of the the software development process, the following functional requirements were defined:

- Application should be able to process any type of photo. (*Req.1*)
- User should be able to create and delete account. (*Req.2*)
- System should store the images in the database. (*Req.3*)
- User should be able to specify the location and filename of the result. (*Req.4*)

3.3 Nonfunctional requirements

In software engineering a nonfunctional requirement defines quality goals of application. It describes how the system works. Before implementation of the software, those nonfunctional requirements were defined:

- Application should be able to run on multiple platforms, as cloud providers are using not only x86_64 CPUs, but also CPUs based on the ARM architecture and z/Architecture. Additionally in recent years new architectures are emerging - it's convenient to have ability to cross-compile the app to Power ISA or RISC-V in the future. (*Req.5*)
- Application should be scalable - it should be easy to add nodes to already working system, as well as to remove them, when the system is not experiencing heavy load. (*Req.6*)
- Processing of image smaller than 150KB should take less than 5 seconds. (*Req.7*)

3.4 Use cases

User must be registered in the system to create any image processing request. To register new user Client application must be run with specific argument combination. User have to specify that he wants to create new account and provide username and passphrase. After the account is created, user can modify passphrase specifying current credentials, new passphrase and the argument showing that he wants to modify the account. Additionally user can request deletion of the account. Argument showing that the user account will be deleted must be provided, as well as the current credentials.

To submit the job, user has to provide credentials of registered account as well as *query* image and *train* image. Additionally user can provide optional flag specifying name of the image as well as optional flag setting the log level.

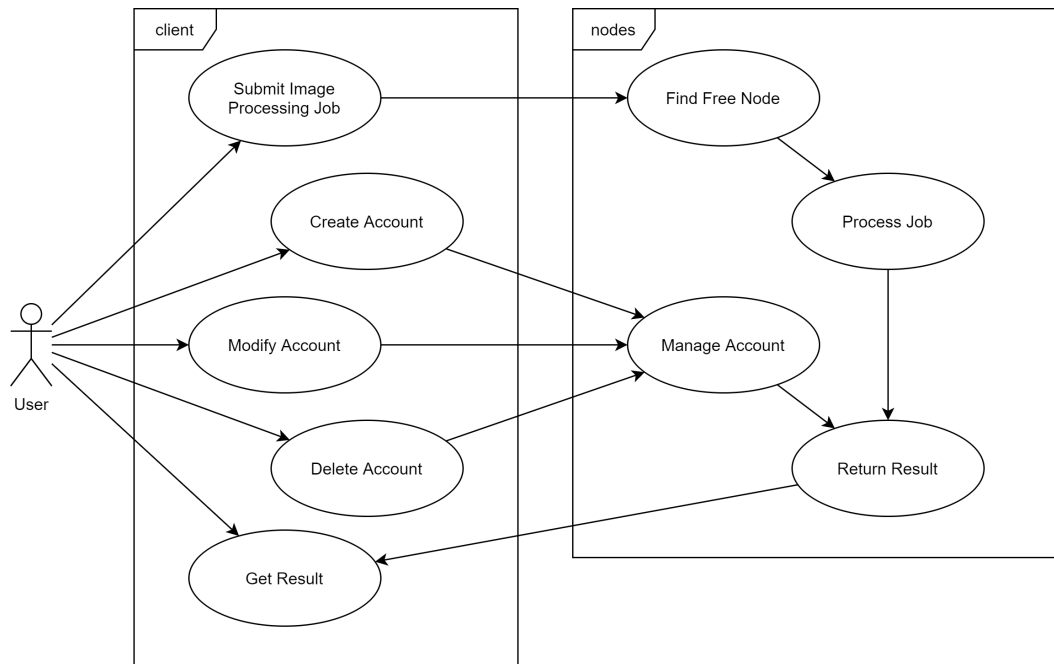


Figure 3.1: Use case diagram.

3.5 Tools

During the project development multiple tools were used. In this section I present them, and shortly describe why they were chosen and how were they used.

3.5.1 Visual Studio Code

Visual Studio Code is a code editor that is easy to extend, supports multiple languages, tools, and platforms. During the development, it was used with many productivity and feature delivering extensions.

- **Go** extension provided extended support for Go programming language and gave access to additional tools built into Go compiler - linter, formatter, importer, and many others.
- **Docker** extension provided support for Docker, including support for linting and formatting container assembly files and Docker-compose configuration files. It also included auto-completion of keywords.

- **Remote - WSL** extension was used in Visual Studio Code on Windows to build and develop the applications inside the Windows Subsystem for Linux (WSL). It allowed combining all features of Windows while developing with Linux-based tools, runtimes, and utilities.
- **Code Spell Checker** was a simple extension that minimized the number of spelling errors and thus minimized the time needed to rebuild the application when such a situation occurs.

3.5.2 Windows Subsystem for Linux

The Windows Subsystem for Linux is a compatibility layer that lets me run a GNU/Linux environment. It included most command-line tools, utilities, and applications directly on Windows, unmodified, without the overhead of a traditional virtual machine or the dual-boot setup. Although the WSL2 is a virtual machine, it is running on Hyper-V Type 1 Hypervisor, which means the performance loss is much smaller compared to running full Linux distribution using Type 2 Hypervisor. It also allowed me to access its file-system directly from Windows Explorer.

3.5.3 protobuf-compiler

protobuf-compiler is a command-line tool used to compile protocol buffer description files. By default, it can generate C++, Objective-C, Java, and Python source code. Additionally, a compilation from a new proto3 description file standard allows extending the capabilities of protobuf-compiler to generate code for Dart, Go, Ruby, and C#.

3.5.4 GNU Make

GNU Make is a utility used to maintain groups of programs and automate the compilation of the source code. During the development, I used it to automate running tests on my local machine, generating code from `.proto` files, compiling the source code, and building and publishing Docker images.

3.5.5 Docker

Docker is set of PaaS products that use OS-level virtualization. It allows delivering of the software in packages called containers. A container is a standard unit of software that packages up code and all its dependencies so the application runs quickly and reliably from one computing environment to another. A Docker container image is a lightweight, standalone, executable package of software that includes everything needed to run an application: code, runtime, system tools, system libraries and settings [5].

3.5.6 Portainer

Portainer is a tool that simplifies container management in Docker, Swarm, Kubernetes, etc. It is deployed on one of them and accessible by a web interface.

3.5.7 pgAdmin 4

pgAdmin 4 is a tool used for the management of PostgreSQL databases. It is available as a container, a macOS desktop application, a Windows desktop application, a Linux desktop application, and a web application using Python.

3.6 Technologies used in the project

In order to achieve the predefined project requirements, the following languages, frameworks, database engine and technologies were used.

3.6.1 Go

Go is an open-source programming language. It was designed to be easy to learn. It focuses on helping developers to build simple, reliable, and efficient software. It is garbage collected, but at the same time, the source code is compiled to binary executable. Its compiler supports cross-compilation out-of-box and was designed to reduce compilation time.

3.6.2 PostgreSQL

PostgreSQL is free and open-source database management engine. Due to its licensing nature it is freely available for possible modification and redistribution, with outstanding community support. It is object-relational - that means it includes features like table inheritance and function overloading. Additionally it includes features that are crucial for development of this system - it can easily store Binary Large Objects in form of Bytea datatype with size up to 1GB.

3.6.3 Shell script

Shell scripts are text files containing a number of commands to be executed by the shell. Scripts can simplify boring, similar, time consuming and repeatable activities.

3.6.4 gRPC framework

gRPC is a modern open source high performance RPC framework. It can efficiently connect services in and across data centers with pluggable support for load balancing, tracing, health checking and authentication. It is also applicable in last mile of distributed computing to connect devices, mobile applications and browsers to back-end services [8].

Chapter 4

Project

4.1 Client-server communication

The client-server communication is done by multiple Remote Procedure Calls. To create those RPCs, gRPC framework was used. The RPCs used in the project can be divided into 2 categories - Job RPC and Account Management RPCs. Both categories will be described in the following subsections.

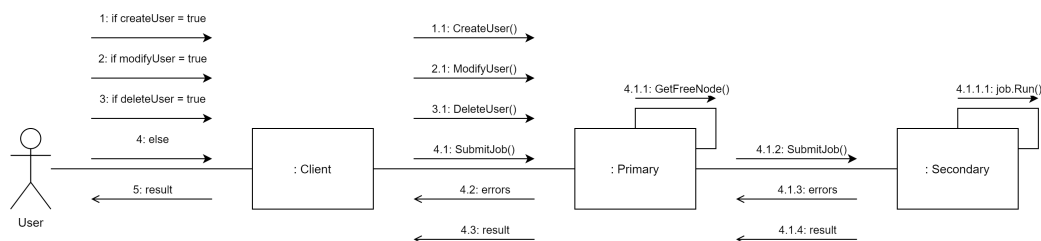


Figure 4.1: Communication diagram of Project.

4.2 Architectural and design patterns

This section describes all architectural and design patterns used in this project. The description is based mostly on *Designing Distributed Systems* book written by Brandon Burns [3] and series of articles written by Unmesh Joshi [11].

4.2.1 Sharded Services Design Pattern

The main idea of the application is to work as sharded service, with Manager Node working as Root or load balancer, and Worker Nodes working as shards or services. With sharded services, each replica, or shard, is only capable of serving a subset of all requests. A load-balancing node, or root, is responsible for examining each request and distributing each request to the appropriate shard or shards for processing [3].

The idea of sharded services is represented in Figure 4.2. The Manager reason for sharding the data is because number of requests is too large to be served by a single computer. Sharding enables you to scale a service in response to the size of the state that needs to be served [15].

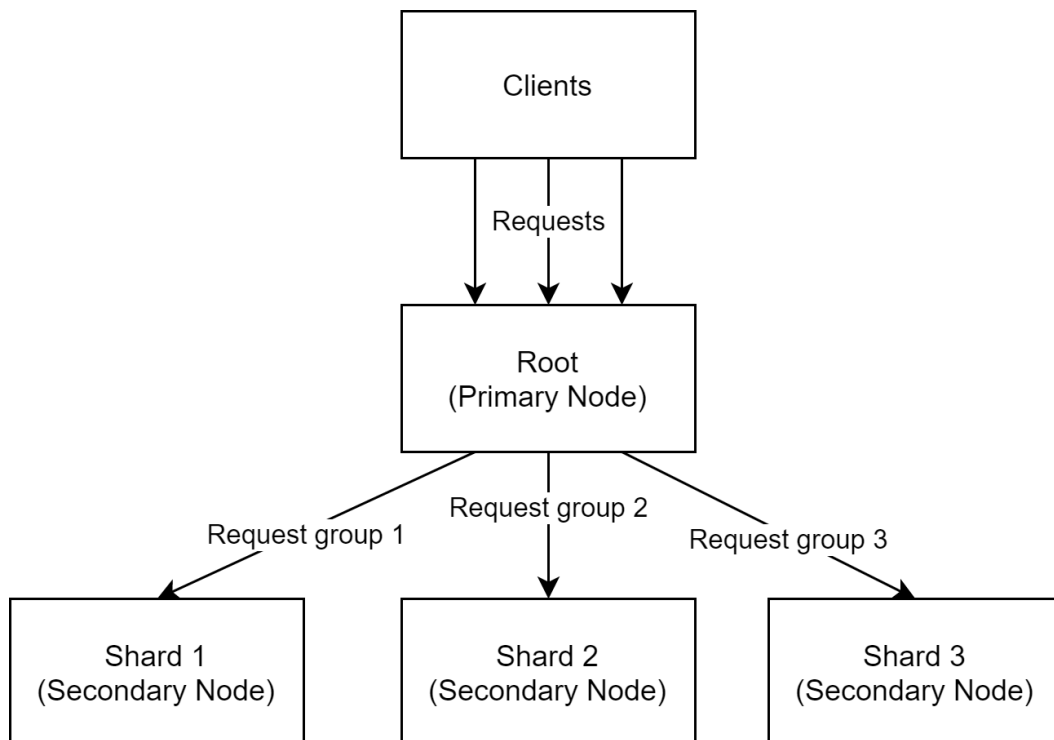


Figure 4.2: Sharded service diagram based on diagram from *Designing Distributed Systems*.

4.2.2 HeartBeat Design Pattern

During the development, I encountered problem of dialing Worker nodes that exited without changing their status in the registry. This was caused most often by receiving *SIGKILL* or *SIGTERM* signal - for example while exiting the container. To fight this, I decided to introduce the HeartBeat design pattern to this project. However instead of sending classic requests to the main server [11], I decided going with the node registry approach. Every 55 seconds the Worker node updates its status to Active in the database, and sets the *node_timestamp* field to the current time and date.

4.3 Remote Procedure Calls

The communication diagram above presents the simplified scheme of the client-server calls. It presents both types of RPCs. More detailed sequence diagrams are presented below.

4.3.1 Job RPC

The Job RPC is the most important part of the application. It takes the data that needs to be processed and returns the result of processing.

The whole processing starts with client dialing the Manager node - the address and port are read from the configuration file. After the connection is established, clients starts the RPC, the stream is created. Then files are being read by client - all basic information about them are sent to the server in the first message. After sending every message, the response is returned. It contains valuable data or - errors.

The file is transported to the Manager Node in chunks. Every message consists of file number and chunk of data that is being transported. After the files are uploaded to the Manager Node, it inserts them into database. After that, Manager Node gets IP address from the database, and simple RPC is called to process the data. Request sends the unique IDs of the files to the Worker Node.

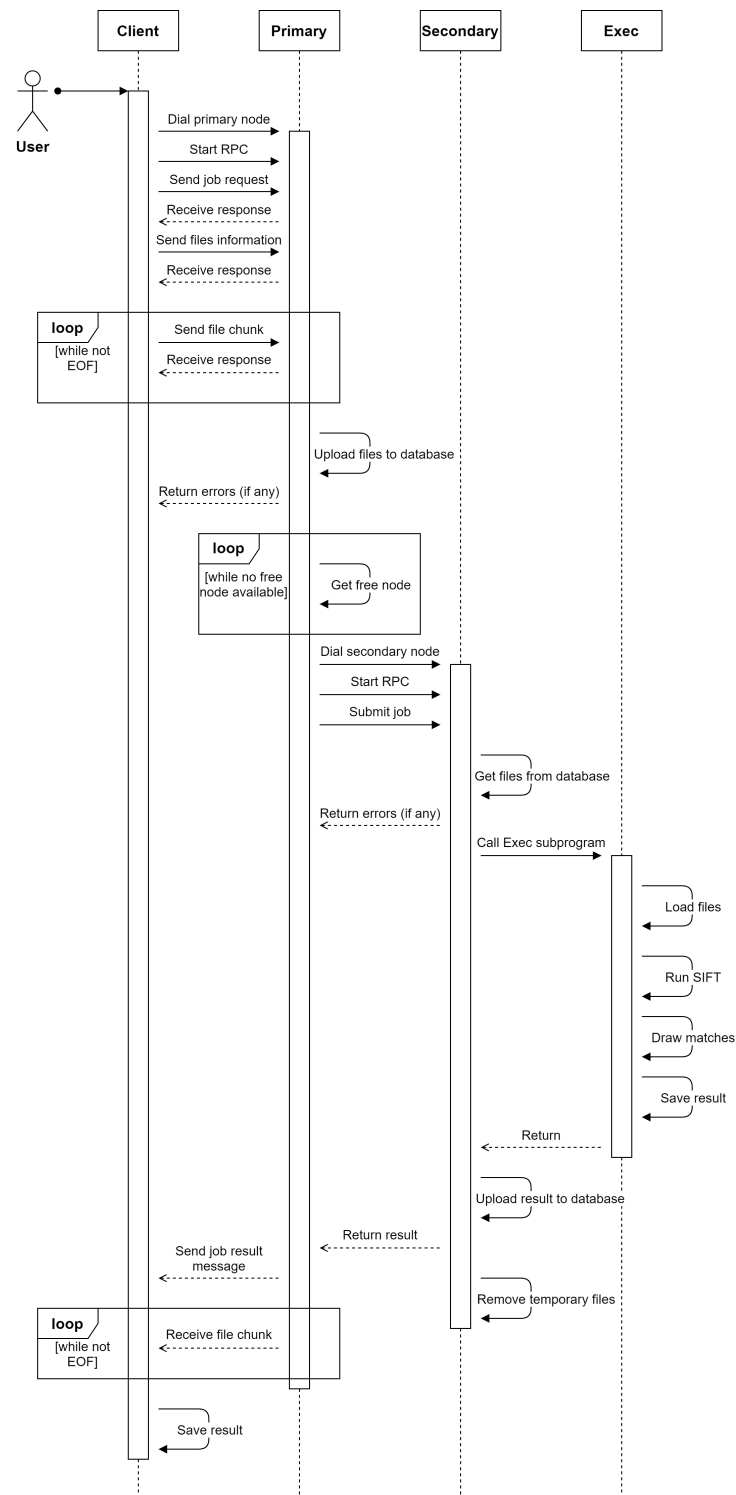


Figure 4.3: Sequence diagram of job RPC

The Worker node downloads files from database, saves it to the local file system and starts subprogram to process them. Subprograms saves the result according to the name passed as one of the arguments to the subprogram during its launch. The result is uploaded to the database, and the ID of the result is sent back to the Manager node, Temporary files are removed from the disk.

After the Worker node returns the response, the Manager node gets the result from database and starts streaming it back to the client.

4.3.2 Caching mechanism

To prevent the obsolete processing of the same combination of files, the caching system was introduced. The client calculates checksums of both files, and sends them with first message to the server. If one of the files is already in the database, this file is skipped, and bandwidth is preserved. In case both of the files are already in the database and this combination was already processed, Manager Node skips the part with the processing the images, and starts sending the result to the client.

4.3.3 Account management RPC

The account management is done by the same client application - the only difference are the flags passed to it on launch. User has to provide current username and passphrase, and in case of modifying the account, additionally provides new passphrase

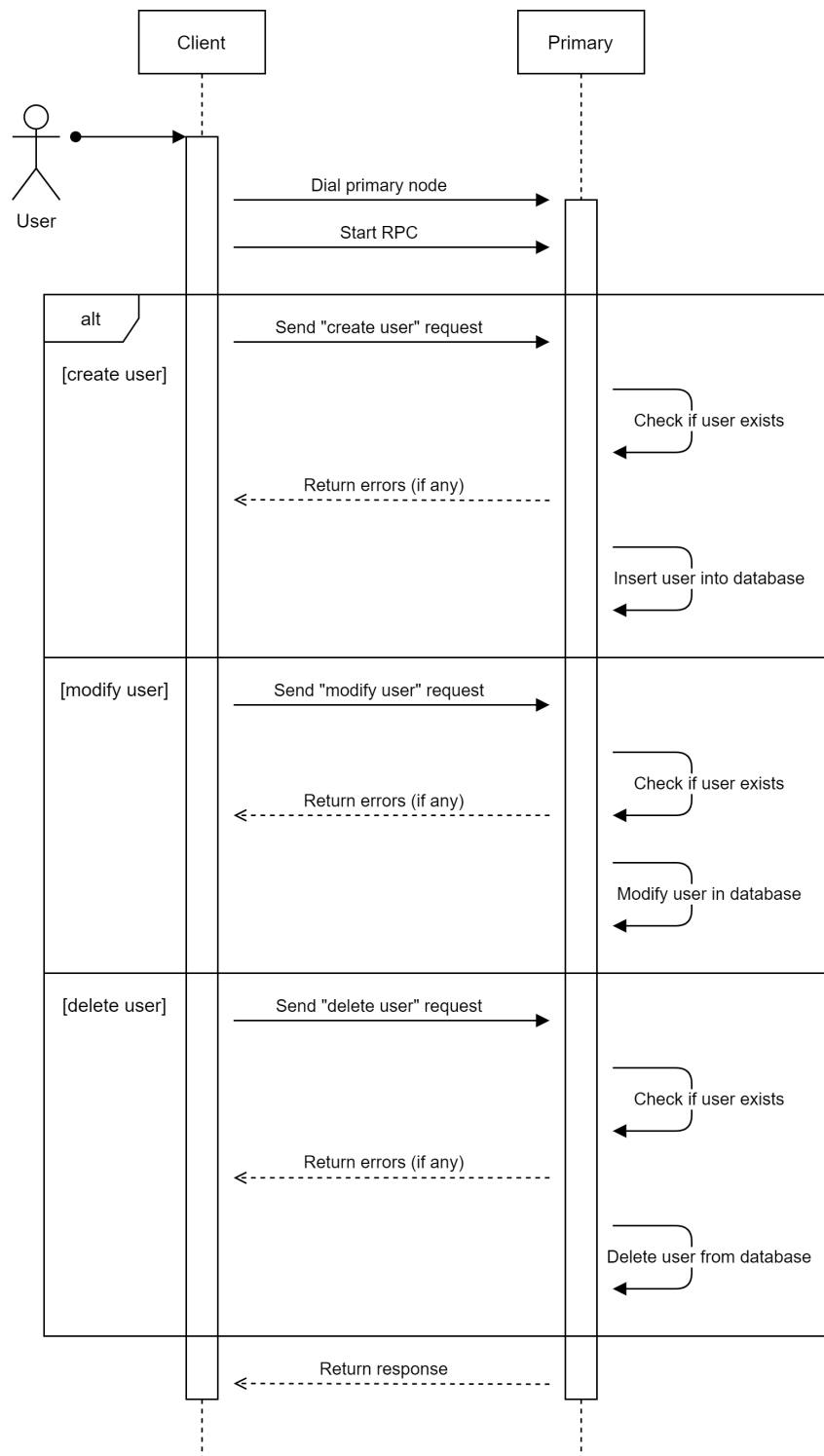


Figure 4.4: Sequence diagram of user management RPCs.

4.4 SQL Database system

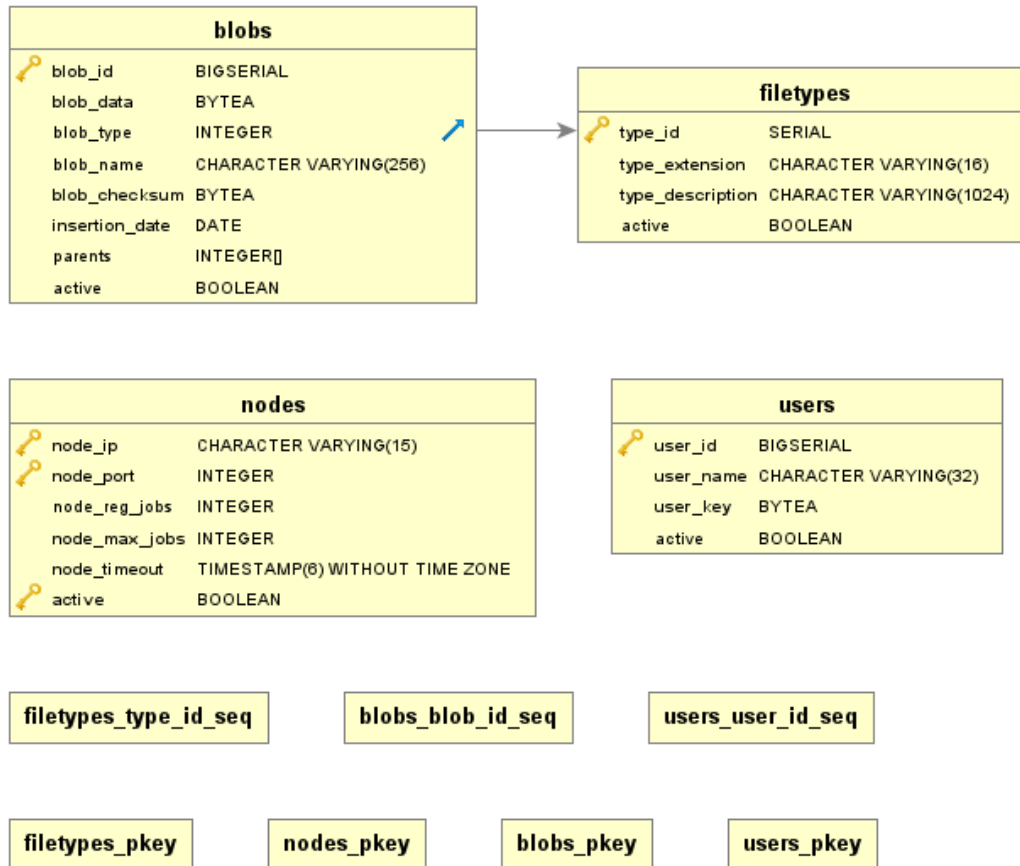


Figure 4.5: Entity Relation Diagram of database used in project.

Database consists of 4 tables. Tables are defined as shown on the Entity Relation Diagram. Additionally, there are defined 3 sequences. For each table is defined PRMIARY KEY.

- **Blobs table** - this is the table designed to hold the binary data of files. It also holds the checksum of each file.
- **Filetypes table** - this table is a dictionary table that holds file extension, identification number and description of each type. Its PRMIARY KEY is referenced inside *blobs* table.

- **Nodes table** - this table holds information about each available node - maximum number of jobs that the node can process, number of currently processing jobs, its address, port, status (active or not) and timestamp used in *Hearbeat* function.
- **Users table** - this table holds the information about the users' accounts - username, encrypted passphrase in binary format, identification number and status.

Chapter 5

Implementation

5.1 System specification

The software was tested on all platforms listed below. Thanks to the use of the Go compiler, it can be easily cross-compile to multiple platforms, and at least the client and the Manager node may be able to run on not officially Supported hardware specifications.

All of the operating systems are determined based on the tests performed on real hardware and inside Proxmox VE Virtualization Management Platform.

Minimum hardware requirements are determined based on the minimum configuration on which the application was run.

5.1.1 Manager Node

Supported platforms

- **Linux** - supported distributions:
 - Ubuntu Server 20.04 LTS x86_64
 - Ubuntu Server 20.04 LTS AArch64
 - Debian 10 (Buster) x86_64
 - Alpine Linux 3.12.0 x86_64
 - CentOS 8 x86_64

- Fedora 33 x86_64
- Ubuntu 20.04 LTS on Windows 10 x86_64 with 4.19.104-microsoft-standard kernel
- RaspberryPi OS AArch64
- **Windows 10** - supported releases:
 - 20H2
 - 1909
- **MacOS** - supported releases:
 - 10.15
 - 11.0

Supported hardware specifications

- **x86_64 / AMD64 platforms:**
 - **CPU:** Intel Core i5-3470 or better
 - **RAM:** At least 1GB
 - **Storage:** At least 20MB for application and 1GB for temporary files
 - **Network card:** At least 100Mbps, preferred 1000Mbps connection through Ethernet cable
- **AArch64 platforms:**
 - **Raspberry Pi 3 Model B** - it is able to run as a Worker, but only for images smaller than 500KB due to limited RAM size.

5.1.2 Worker Node

Supported platforms

- **Linux** - tested distributions:

- Ubuntu Server 20.04 LTS x86_64
- Ubuntu Server 20.04 LTS AArch64
- Debian 10 (Buster) x86_64
- Alpine Linux 3.12.0 x86_64
- CentOS 8 x86_64
- Fedora 33 x86_64
- Ubuntu 20.04 LTS on Windows 10 x86_64 with 4.19.104-microsoft-standard kernel
- RaspberryPi OS AArch64

Supported hardware specifications

- **x86_64 / AMD64 platforms:**
 - **CPU:** Intel Core i5-3470 or better
 - **RAM:** At least 1GB per CPU Core when processing files smaller than 700KB. For bigger files, consider using 4GB per CPU Core, as the SIFT matching is memory heavy,
 - **Storage:** At least 20MB for application and 1GB for temporary files
 - **Network card:** At least 100Mbps, preferred 1000Mbps connection through Ethernet cable
- **AArch64 platforms:**
 - **Raspberry Pi 3 Model B** - it is able to run as a Worker, but only for images smaller than 500KB due to limited RAM size.

Additional software and libraries

- **OpenCV** v4.5.0

5.1.3 Client

Supported platforms

- **Linux** - supported distributions:
 - Ubuntu Server 20.04 LTS x86_64
 - Ubuntu Server 20.04 LTS AArch64
 - Debian 10 (Buster) x86_64
 - CentOS 8 x86_64
 - Fedora 33 x86_64
 - Ubuntu 20.04 LTS on Windows 10 x86_64 with 4.19.104-microsoft-standard kernel
 - RaspberryPi OS AArch64
- **Windows 10** - supported releases:
 - 20H2
 - 1909
- **MacOS** - supported releases:
 - 10.15
 - 11.0

Hardware requirements

- **CPU:** Intel Celeron J1800 or better
- **RAM:** At least 1GB
- **Storage:** At least 20MB for application and additional space for result file
- **Network card:** At least 100Mbps

5.1.4 Database

As PostgreSQL is external part of the application, the requirements for it are available on the official documentation page.

In general, PostgreSQL can be expected to work on these CPU architectures: x86, x86_64, IA64, PowerPC, PowerPC 64, S/390, S/390x, Sparc, Sparc 64, ARM, MIPS, MIPSEL, M68K, and PA-RISC. Code support exists for M32R and VAX, but these architectures are not known to have been tested recently. It is often possible to build on an unsupported CPU type by configuring with `-disable-spinlocks`, but performance will be poor.[7]

PostgreSQL can be expected to work on these operating systems: Linux (all recent distributions), Windows (Win2000 SP4 and later), FreeBSD, OpenBSD, NetBSD, OS X, AIX, HP/UX, Solaris, and UnixWare. Other Unix-like systems may also work but are not currently being tested. In most cases, all CPU architectures supported by a given operating system will work. Look in the Section 15.7 below to see if there is information specific to your operating system, particularly if using an older system.[7]

5.2 Manager Node

Manager node holds two main functions - it is load balancer and proxy between client and worker. It implements all types of remote procedure calls - internal job forwarding, user management and external job acquirement. Two main coroutines (in Go those lightweight threads are named goroutines) with watchdog and gRPC server are working while main thread is used just in the configuration stage.

5.3 Worker Node

Worker implements only one type of remote procedure call - internal job RPC. It has only one purpose - to receive job request and processes it on one of threads. Maximal number of worker threads are defined in the configuration. After reading configuration file or environmental variables, main thread is suspended and three coroutines are called - heartbeat, watchdog and gRPC server.

5.3.1 HeartBeat

HeartBeat pattern is implemented as goroutine. This function (shown in 5.1) updates record with the timestamp in the database. If update fails, it retries that update up to 5 times. If the update fails 5 times, then the fatal error is raised, and the whole Node crashes.

```

1 func Heartbeat(conf convo.Config) {
2     var err error
3     count := 1
4     for {
5         err = db.UpdateTimestamp(context.Background(),
6             conf)
7         if err != nil {
8             if count <= 5 {
9                 plog.Errorf("error #%v while updating
10                     timestamp: %v", count, err)
11                 count++
12                 time.Sleep(15 * time.Millisecond)
13             } else {
14                 plog.Fatalf(codes.WorkerConnectionError,
15                     "fatal error while updating timestamp: %v
16                     ", err)
17             }
18         } else {
19             plog.Debug("heartbeat updated")
20             time.Sleep(50 * time.Second)
21             count = 1
22         }
23     }
24 }
```

Figure 5.1: Function that is run as subroutine for updating the timestamp in the database.

5.3.2 Image Processing

The image processing is done by calling subprogram from the Worker Node. In this case, the subprogram is defined using `exec.CommandContext()`, which allows

to pass context to the execution code. Three arguments to the command are passed - query image name, train image name and output image name. This allows to swap the subprogram without recompiling the whole Worker Node server, as long as the program takes the same arguments as the original subprogram. Additionally, use of the *Context* allows to cancel the job immediately after user cancels his job request, without waiting for calculations to finish.

5.4 Client

Client is simple program accessed by terminal or command prompt with the CLI. Options are passed with flags (also known as command line arguments). All available flags are presented bellow.

- `-user-new` - defined if user wants to create new account, cannot be used with `-user-del` or `-user-mod` flags, must be followed by `-uname` and `-pwd` flags;
- `-user-del` - defined if user wants to delete existing account, cannot be used with `-user-new` or `-user-mod` flags, must be followed by `-uname` and `-pwd` flags;
- `-user-mod` - defined if user wants to modify existing account, cannot be used with `-user-new` or `-user-del` flags, must be followed by `-uname`, `-pwd` and `-pwd-new` flags;
- `-uname` - specifies username for account;
- `-pwd` - specifies password for account;
- `-pwd-new` - specifies new password while modifying account;
- `-source-img1` - defines path to the 1st image that has to be processed;
- `-source-img2` - defines path to the 2nd image that has to be processed;
- `-o` - defines path and/or file name to which result will be saved, should not contain file extension;

Exemplary command to run the application:

```

1 client.run -uname=user -pwd=password \
2   -source -img1=img1.png -source -img2=img2.jpg -o=
   result

```

5.5 Image processing subprograms

Subprogram responsible for image processing can be easily swapped for another, as long as both are taking the same parameters and returning the same result. Available parameters for the subprogram are presented below.

- `-img` - name of the input file, is variable argument list, that means it can be repeated multiple times;
- `-out` - name of the output file - should not contain extension;

Full listing of *BestMatches* is available on page XI.

```

1 outMat := compute.BestMatches(*img1 , *img2)
2
3 compute.SaveMat(*out , outMat)

```

Figure 5.2: Part of computation subprogram responsible for computation.

Program performs SIFT on both images. *BestMatches* takes two arguments being *Query* and *Train* and holding paths to the images that have to be processed during the SIFT detection and Brute force matching. Function returns *gocv.Mat* holding result of feature matching algorithm. Two files specified by flags are being loaded from the file system. After that SIFT analysis is performed, and keypoints are detected and descriptors are computed. After that, Brute Force matching is performed - finding K best matches for each descriptor (in case of this program 2, to be able to perform Lowe's ratio test [13]). Before drawing matches, the Lowe's ratio test is performed, with coefficient of 0.75. Each keypoint of the first image is matched with a number of keypoints from the second image. The 2 best matches

for each keypoint are kept (best matches means matches with the smallest distance measurement). Lowe's test checks that the two distances are sufficiently different. If they are not, then the keypoint is eliminated and will not be used during drawing matches.

5.6 Deployment process

This section contains description of the deployment process.

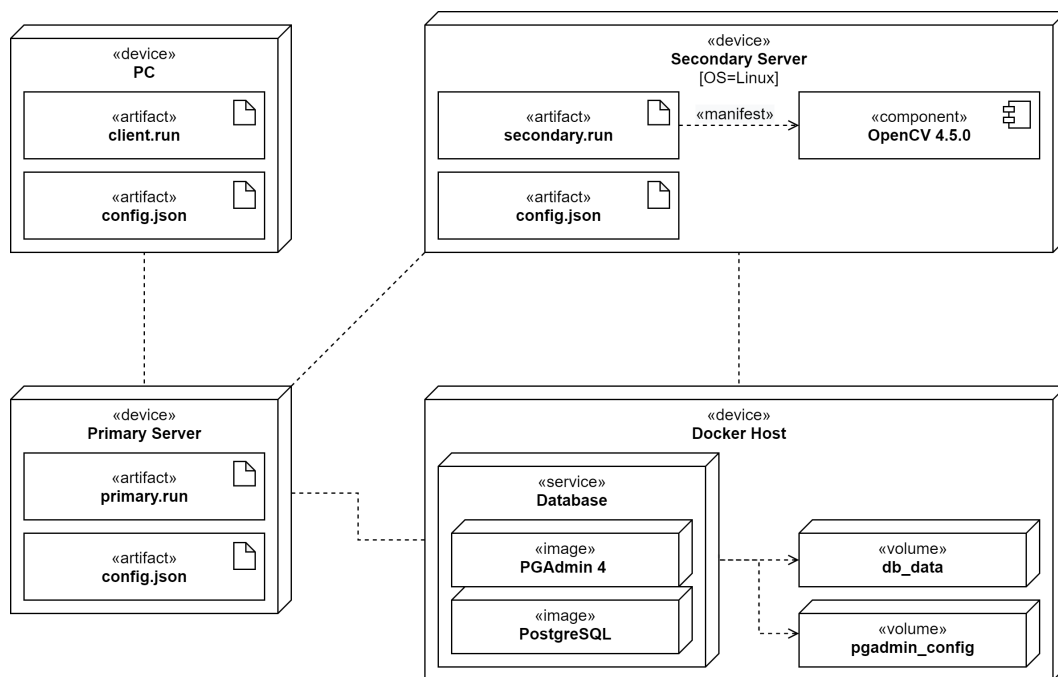


Figure 5.3: Deployment diagram of application.

5.6.1 Docker and docker-compose

The easier way to deploy the application is using Docker and Docker-Compose. Prebuilt images are freely available on the Docker Hub. To deploy it on single machine, system administrator have to write deployment configuration file.

Configuration of Docker Compose consists mainly on setting environmental variables holding configuration like Node address and external port.

```

1 version: 2.4
2
3 services:
4   manager:
5     image: shanduur/pluggabl-Manager
6     restart: unless-stopped
7     environment:
8       - TZ=Europe/Warsaw
9       - PG_DATABASE=postgresql://192.168.1.100:5432/
10         dbname?user=admin&password=password
11     ports:
12       - '4004:4004'
13
14   worker:
15     image: shanduur/pluggabl-Worker
16     restart: unless-stopped
17     environment:
18       - TZ=Europe/Warsaw
19       - PG_DATABASE=postgresql://192.168.1.100:5432/
20         dbname?user=admin&password=password
21       - EXTERNAL_PORT=4010
22       - EXTERNAL_IP=192.168.1.101
23       - LOG_LEVEL=info
24     ports:
25       - '4010:4004'

```

Figure 5.4: The docker-compose deployment description file example.

In this file, administrator has to define services. Manager stands for the Manager node, while Worker is the Worker node. All available configuration options are available below.

- **EXTERNAL_IP** - sets the external IP on which the container is accessible (IP of the host).
- **EXTERNAL_PORT** - sets the external port on which the container is accessible (port on the host). The same port have to be used in the *ports* section

of the deployment file. By default it is set to 4010 for Worker and 4004 for Manager.

- **GC_PERCENTAGE** - sets the garbage collection target percentage: a collection is triggered when the ratio of freshly allocated data to live data remaining after the previous collection reaches this percentage. Cannot be set to value lower than 100. Default value is 1000.
- **LOG_LEVEL** - defines how detailed logs are. Default value when it is not set is *warning*. Possible values:
 - verbose
 - debug
 - info
 - warning
 - error
 - fatal
- **THREADS** - defines how many worker threads can be processed on single node. Default is 4.
- **TZ** - sets the current timezone. It is crucial, as the nodes have to live in the same timezone - it is necessary for the Heartbeat 5.1. By default it is set to UTC.

After creating deployment description file, administrator have to issue the following command in command line / terminal open in the location of the file:

```
1 docker-compose up --detach
```

5.6.2 Bare-metal deployment

This is the more complicated way to deploy the application. To do that, the system must have installed the Go compiler, the Git client and the GNU Make.

Additionally on the Fedora and CentOS additional issues might occur. Solution for most of them are located in the CentOS and Fedora troubleshooting section.

In the beginning, system administrator have to clone the project to the machine:

```
1 git clone https://github.com/Shanduur/pluggabl
```

Next, if system administrator wants to build OpenCV and use every part of the software, he have to issue the following command to build all the necessary libraries:

```
1 make prebuild
```

Next, on the non-Unix system to build binaries for the Manager node and Client, system administrator have to issue the command:

```
1 make build
```

On the Unix-like system to build all the binaries system administrator have to issue the command:

```
1 make build_all
```

Finally, on the Unix systems, you can install it directly from the Makefile. To do that, system administrator have to issue the following command, during which, he might be prompted for the password:

```
1 make install
```

After installation, it might be necessary to reload the shell. After that you can run the following command to obtain additional help:

```
1 pluggabl -help
```

On non-Unix hosts, you have to manually create the following directory tree:

```
1 C:\users\UserName\.config\pluggabl.d\
```

In the directory, you have to create file for the client: *config_client.json*.

5.6.3 Configuration Files Examples

Every part of the application when run on the bare-metal, need configuration file. It can be located in the default location (`~/.config/pluggabl.d` for client and `~/opt/pluggabl/` for both types of nodes), or it's location can be provided through environmental variable `CONFIG` independently of platform.

1. Client

```
1 {  
2     "type": "client",  
3     "address": "192.168.1.101",  
4     "port": 4004  
5 }
```

Figure 5.5: Example of Client configuration file.

2. Manager

```
1 {  
2     "type": "primary",  
3     "address": "0.0.0.0",  
4     "port": 4004,  
5     "external-port": 4004,  
6     "garbage-collection-timeout": 1000  
7 }
```

Figure 5.6: Example of Manager node configuration file.

3. Worker

```
1 {  
2     "type": "secondary",  
3     "job-binary-name": "/pluggabl/exec.run",  
4     "address": "192.168.1.110",  
5     "port": 4005,  
6     "external-port": 4005,  
7     "garbage-collection-timeout": 1000,  
8     "max-threads": 4  
9 }
```

Figure 5.7: Example of Worker node configuration file.

4. Database Connection

```
1 {  
2     "type": "db",  
3     "address": "192.168.1.120",  
4     "port": 5432,  
5     "database-name": "pluggable_db",  
6     "database-username": "admin",  
7     "database-password": "password"  
8 }
```

Figure 5.8: Example of database connection configuration file.

5.6.4 CentOS and Fedora troubleshooting

libdc1394-devel not found

In the CentOS 8 administrator must additionally install third party raven-release repository and install few packages by hand:

```
1 sudo dnf install https://pkgs.dyn.su/el8/base/x86_64/  
    raven-release-1.0-1.el8.noarch.rpm  
2 sudo dnf --enablerepo=PowerTools install libraw1394 -  
    devel
```



```
3 sudo dnf --enablerepo=raven-multimedia install libdc1394
  -devel
```

Missing CXX compiler

On both Fedora 33 and Centos 8, user have to install Clang compiler:

```
1 sudo dnf install clang
```

Missing package opencv4

After installing the OpenCV 4.5.0 on the CentOS 8 using command `make prebuild`, user might get error:

```
1 # pkg-config --cflags -- opencv4
2 Package opencv4 was not found in the pkg-config search
  path.
3 Perhaps you should add the directory containing 'opencv4
  .pc' to the PKG_CONFIG_PATH environment variable
  Package 'opencv4', required by 'virtual:world', not
  found
4 pkg-config: exit status 1
```

To fix this error, symbolic link to the *opencv4.pc* file must be created. The file should be in the */usr/local/lib64/pkgconfig/opencv4.pc* folder. User have to issue the following command to fix the problem:

```
1 sudo ln -s /usr/local/lib64/pkgconfig/opencv4.pc /usr/
  share/pkgconfig/
```

Fedora: Cannot open shared object file

During build on Fedora 33 user may encounter the following problem:

```
1 ./build/linux/amd64/exec.run: error while loading shared
   libraries: libopencv_gapi.so.4.5: cannot open shared
   object file: No such file or directory
```

To fix it, the following commands have to be issued:

```
1 sudo bash -c "echo /usr/local/lib64/ > /etc/ld.so.conf.d
   /opencv4.5.0.conf"
2 sudo ldconfig
```

5.6.5 Database deployment

Deployment of database management engine can be done through Docker and Docker-Compose or bare-metal. The installation process is widely described in the PostgreSQL documentation [6] or on the official PostgreSQL Docker-Hub page, After installing the database management engine, administrator should connect to it and create database according to the ERD.

Chapter 6

Verification and validation

6.1 Methodology of testing

During the development V model was used for testing. After analysis and design stages, all parts of applications were tested. Verification of fulfilling all requirements are presented in the Table 6.1.

Table 6.1: Fulfilled Requirements

Requirement	Reference	Fulfilled
Functional testing		
Application should be able to process any type of image	Req.1	Yes
User should be able to create and delete account	Req.2	Yes
System should store the images in the database	Req.3	Yes
User should be able to specify the location and filename of the result	Req.4	Yes
Non-functional testing		
Application should be able to run on multiple platforms	Req.5	Yes
Application should be scalable	Req.6	Yes
Processing of image smaller than 150KB should take less than 5 seconds	Req.7	Yes

Parts of the program were tested using Unit Test methodology. Unit tested were configuration package, logging package and serialization package. After performing

unit tests, integration tests were performed. After deployment of the application I prepared scripts, that can be run in PowerShell or Bash. With those scripts, I gathered the data that can be seen in the following section.

6.2 Testing results

Client and processing subprogram implemented and described in this project, performed brute force feature matching with extraction of features done by SIFT feature detection algorithm [13]. The result of processing is output image, but the output format is not specified by API, but by processing subprogram.

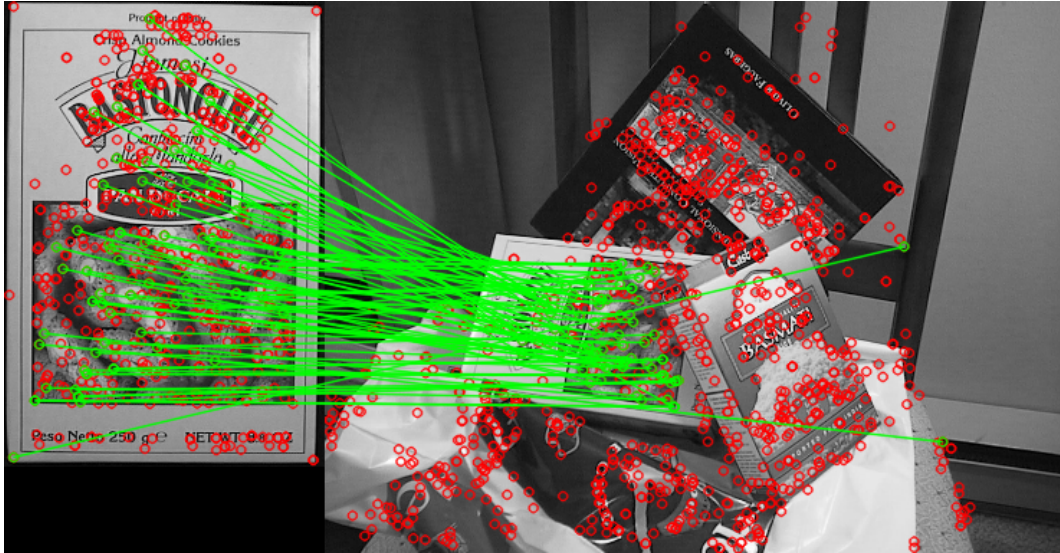


Figure 6.1: Result of image processing on OpenCV's exemplary images, *box* and *box_in_scene* using default exec program.

6.2.1 Results - 80 small files

The results of performance tests are available in the table below. First column describes amount of threads used in this part of testing, *Elapsed time* is the amount of time that was spent on processing number of images described by following equation: $I = \frac{N}{T} \cdot N$, where I is a number of images per thread, N is a number of images in the set, and T is a number of threads used during the image processing.

User time was the amount of time the CPU was busy executing code in user space of Client, while the *System time* is the amount of time the CPU was busy executing code in kernel space of Client.

The Table 1: *Performance results of system processing 80 files* (can be found on XIV in appendix) presents the results of processing 80 files on the system deployed on single Docker host with 20GB of RAM and 20 threads. 4 workers with 4 threads each were deployed in this scenario. Size of each file was under 780KB.

6.2.2 Results - 4 large files and Docker

The Table 6.2 presents the results of processing 4 files on the system deployed on single Docker host with 20GB of RAM and 20 threads. 4 workers with 4 threads each were deployed in this scenario. Each file's size was ranging from 1.8MB to 2MB. Elapsed time format is little bit different, as it includes additional milliseconds field. It can be observed that with the number of cores elapsed time increased instead of decreasing.

Table 6.2: Performance results of system processing 4 files using cache mechanism from beginning.

Threads	Elapsed time	User time	System time
1	00:15:36.770	19.17	8.70
2	00:14:31.070	10.29	5.38
2	00:15:30.960	8.07	3.91
4	00:22:18.700	4.59	1.93
4	00:20:37.340	4.57	2.33
4	00:21:40.200	4.25	2.08
4	00:17:05.680	6.55	3.46

6.2.3 Results - 4 large files and baremetal

To obtain another set of results, the Workers were deployed once again on another set of machines:

- Fedora 33 - 2 cores, 4GB of RAM
- CetnOS 8 - 2 cores, 4GB of RAM

- Debian 10 - 2 cores, 4GB of RAM
- Docker on Ubuntu 20.04 - 2 cores, 4GB of RAM

Table 6.3: Performance results of system processing 4 files using cache mechanism from beginning and with changed deployment method.

Threads	Elapsed time	User time	System time
1	01:29:27.000	20.89	9.43
2	00:50:50.410	8.66	4.27
2	00:43:46.540	11.24	5.32
4	00:31:35.220	4.65	2.12
4	00:28:57.160	5.23	1.96
4	00:29:55.930	4.77	2.03
4	00:18:29.720	6.91	3.04

6.3 Results analysis

6.3.1 Performance increase - 80 small files

The *Elapsed time* results from the Table 1 can be simplified to *Average elapsed time*, as shown in the Table 6.4. As we can see clearly now, the biggest improvement is obtained while changing from single thread to two threads. Despite the fact, that the improvement between 8 and 16 threads was not that huge compared to 1 and 2 threads, it was still noticeable.

Table 6.4: Comparison of the number of threads and average elapsed time during processing of 80 files.

Threads	Average elapsed time
1	4:13:32
2	2:09:35
4	1:29:17
8	1:13:15
16	1:03:27

The plot in the Figure 6.2 compares the ideal improvement with actual obtained during testing. Error bars visible on the plot are differences between average processing time and maximal and minimal processing times, as the processing times

differed between each others. The linear improvement was calculated using naive formula:

$$t_n = \frac{t_{n-1}}{2}$$

This linear results cannot be obtained in this case.

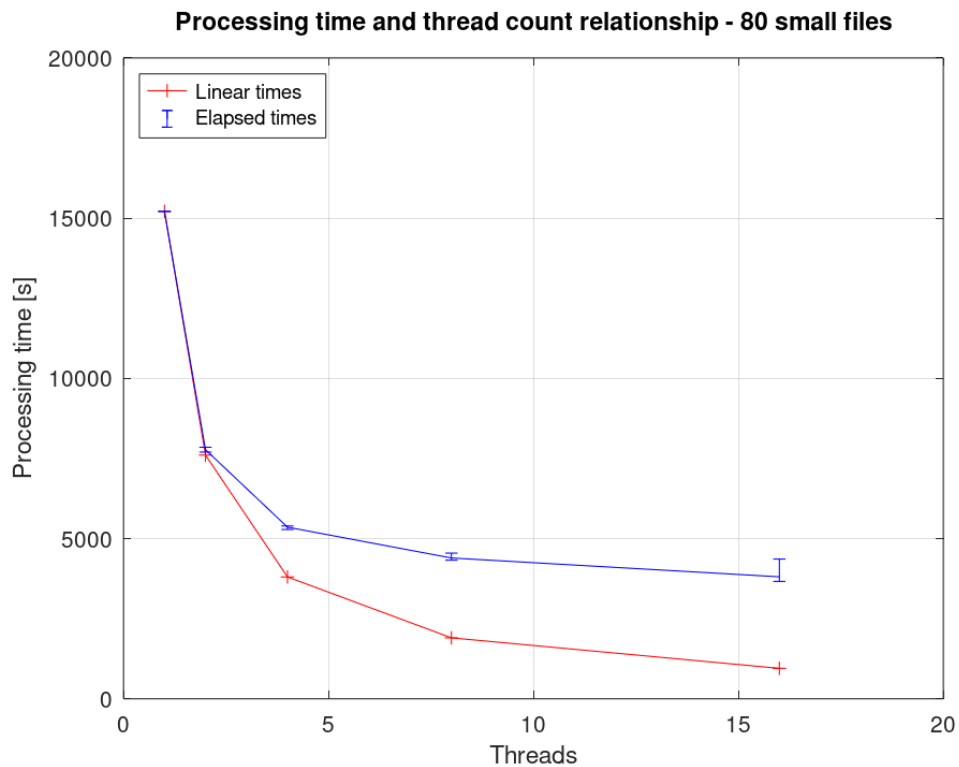


Figure 6.2: Plot showing timing decrease as a result of adding Workers, compared to unobtainable ideal solution.

The speedup (visible in the Figure 6.3) calculated from the processing time gathered during the testing, is far from linear, which should be the goal. This is caused by the network overhead as well as the disk I/O operations limit. To mitigate impact of the network, better networking gear should be used. Additionally, to improve disk I/O operations speed, a SSD should be used instead of the HDD.

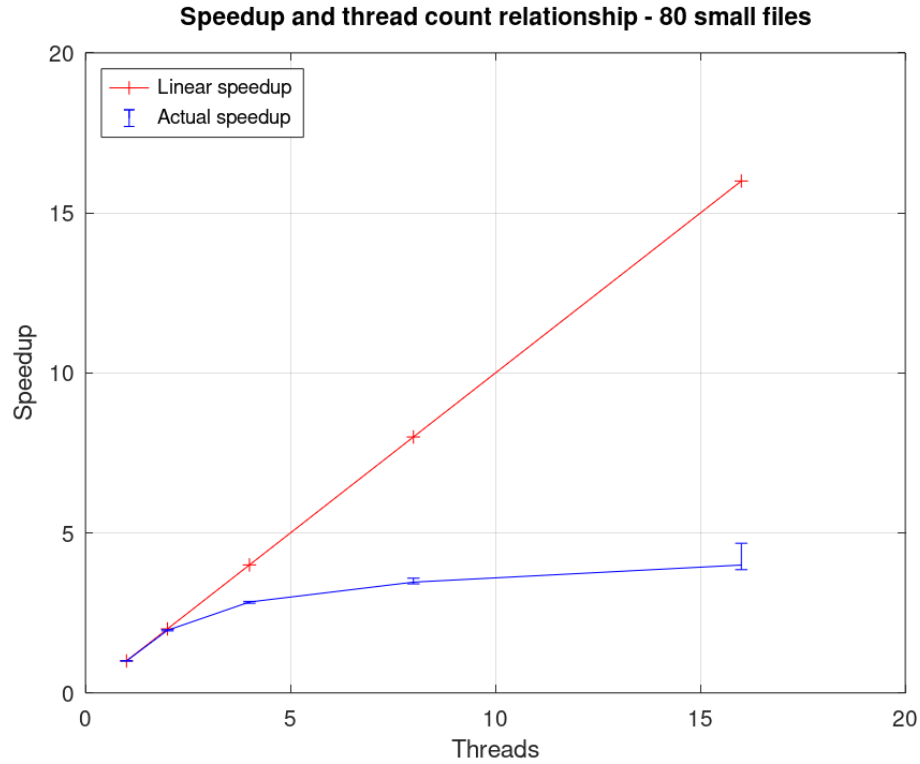


Figure 6.3: Plot showing speedup in processing of 80 files.

6.3.2 Performance decrease - 4 large files and Docker

The results from the Table 6.2 were simplified into the Table 6.5. This table shows, that improvement between 1 and 2 threads in this particular case was negligible. The reason why the results obtained were that strange, is due to the way, how the application was deployed.

Table 6.5: Comparison of the number of threads and average elapsed time during processing of 4 files.

Threads	Average elapsed time
1	00:15:36.770
2	00:15:01.015
4	00:20:25.480

The deployment of the application on a single Docker host combined with the

OpenCV library built with multithreading support caused usage of every single processor thread available by every single Worker's thread. This caused CPU scheduler to switch between all Worker's threads frequently. This introduced additional latency to the system.

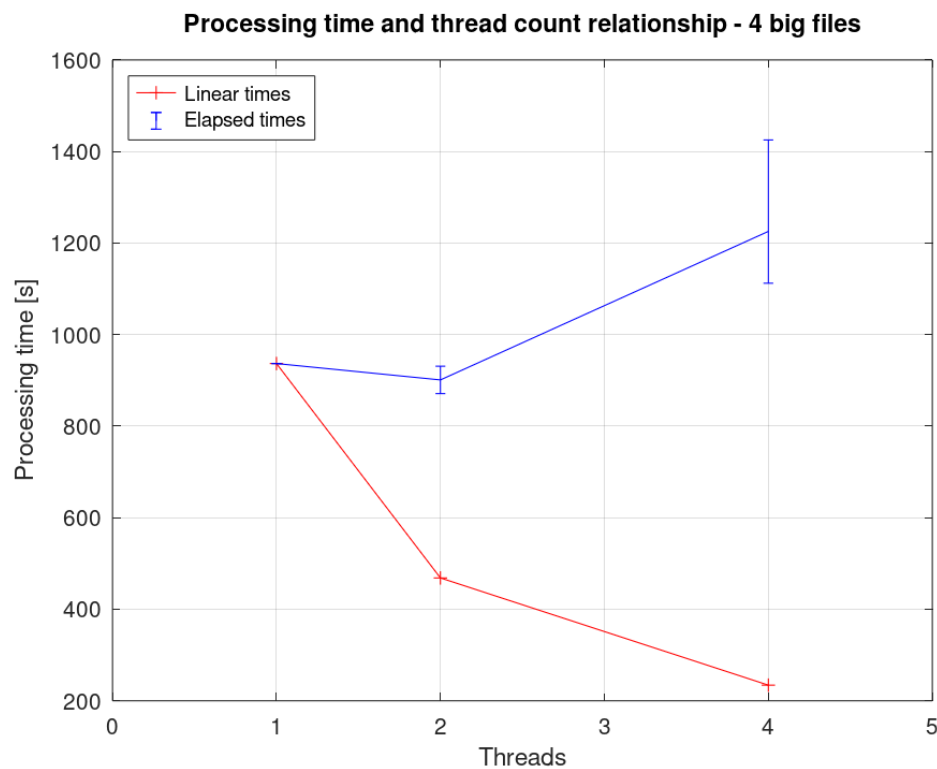


Figure 6.4: Plot showing timing increase, compared to unobtainable ideal solution.

Besides the performance decrease, it can also be observed that the error margin is huge. With small amount of files, threads are not equally loaded, and all the differences in file sizes picked up by threads are visible.

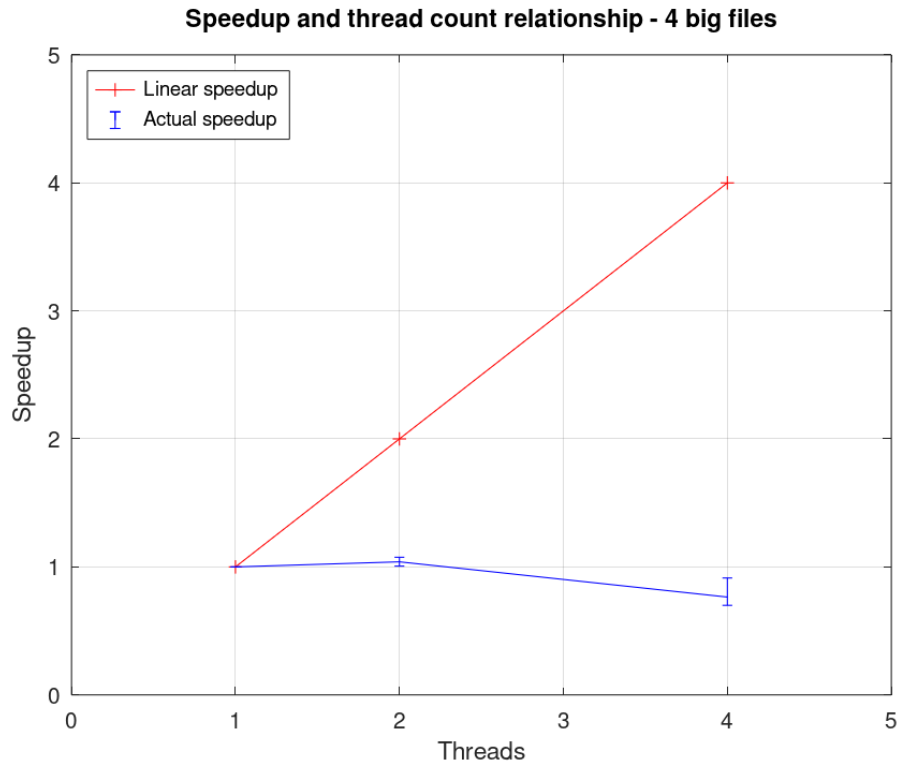


Figure 6.5: Plot showing speedup in processing of 4 big files.

6.3.3 Performance increase - 4 large files and baremetal

After deploying the Workers on the physical machines instead of Docker, the obtained results were much more predictable.

Table 6.6: Comparison of the number of threads and average elapsed time during processing of 4 files after changing the deployment method.

Threads	Average elapsed time
1	01:29:27.000
2	00:47:18.475
4	00:27:14.508

Processing files with sizes varying from 1.8MB to 2MB, on properly deployed system, we can observe nearly linear performance increase. One of the threads actually achieved the time equal to time calculated for the linear speedup, with

error margin less than 1%.

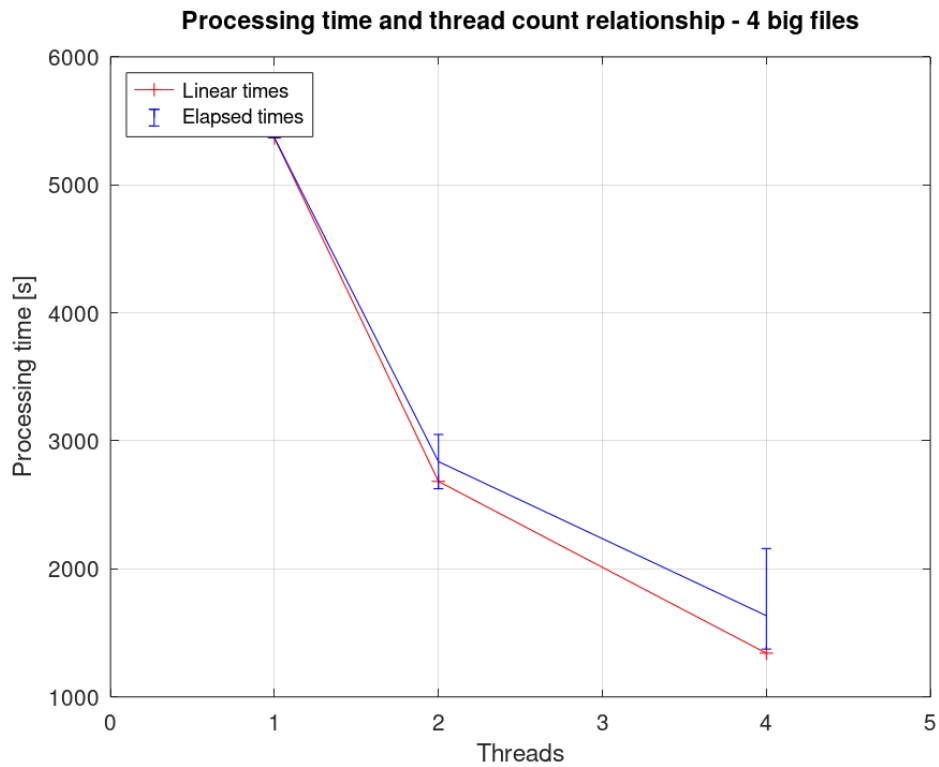


Figure 6.6: Plot showing timing decrease, compared to unobtainable ideal solution.

Unfortunately errors visible in the speedup graph are much more prominent. This was caused most likely by the disproportion between workstations performance.

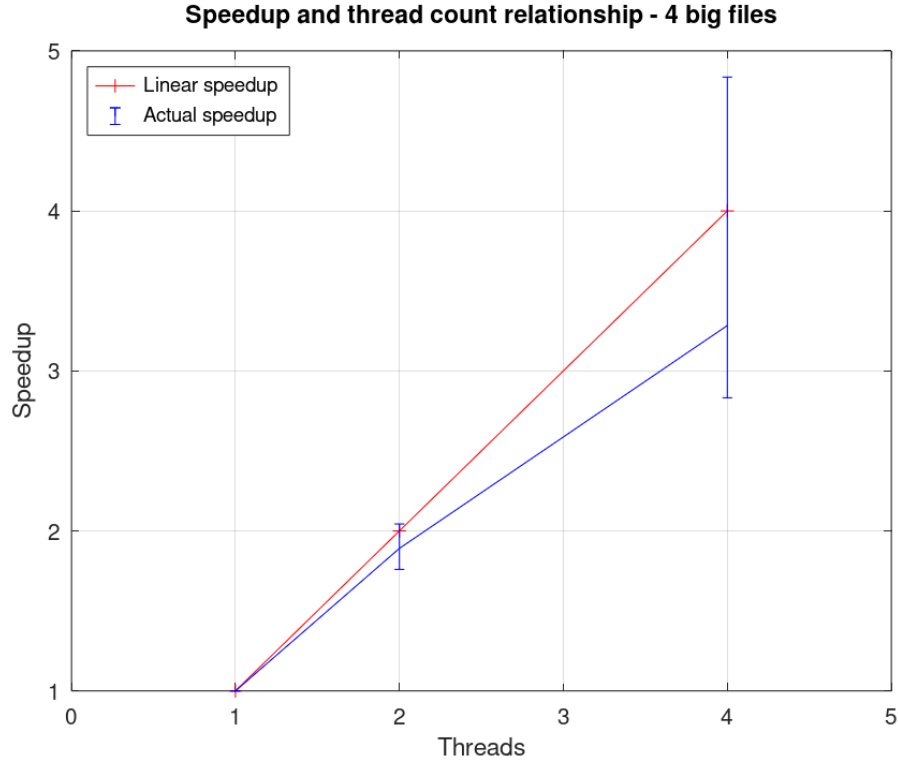


Figure 6.7: Plot showing speedup in processing of 4 big files after changing the deployment method.

6.3.4 Speedup analysis using Amdahl's law

Amdahl's law [10] is a formula which gives the theoretical speedup in latency of the execution of a task at fixed workload that can be expected of a system whose resources are improved.

Applying Amdahl's law to the dataset of 80 files, we can calculate and compare theoretical results with the obtained ones.

$$S_{task}(s) = \frac{1}{(1 - p) + \frac{p}{s}}$$

Where

- S_{task} is the theoretical speedup of the whole task;
- s is the number of threads across which the parallel portion is split;

- p is the proportion of execution time that the part benefiting from improved resources originally occupied.

p is estimated as number of images that are processed minus one divided by number total count of images. It is assumed, that the execution time of the part that would not benefit from the improvement of the resources is equal to processing time of single image and the execution time of the part that would benefit from it are all other images that are included in the set.

Table 6.7: Comparison of the theoretical and actual speedup of processing 80 images

Threads	Theoretical speedup	Actual speedup	Difference
1	1.0000	1.0000	0
2	1.9753	1.9565	-0.0188
4	3.8554	2.8398	-1.0156
8	7.3563	3.4613	-3.8950
16	13.4737	3.9957	-9.4780

The Table 6.7 shows, that obtained results are far from ideal. As I stated before, this is caused mainly by network overhead and the fact that OpenCV uses multithreading by default. Adding worker threads introduces parallelisms to image processing, but at the same time increases amount of time needed to process single image in this particular scenario.

We can observe in the Figure 6.8 how significant the difference is.

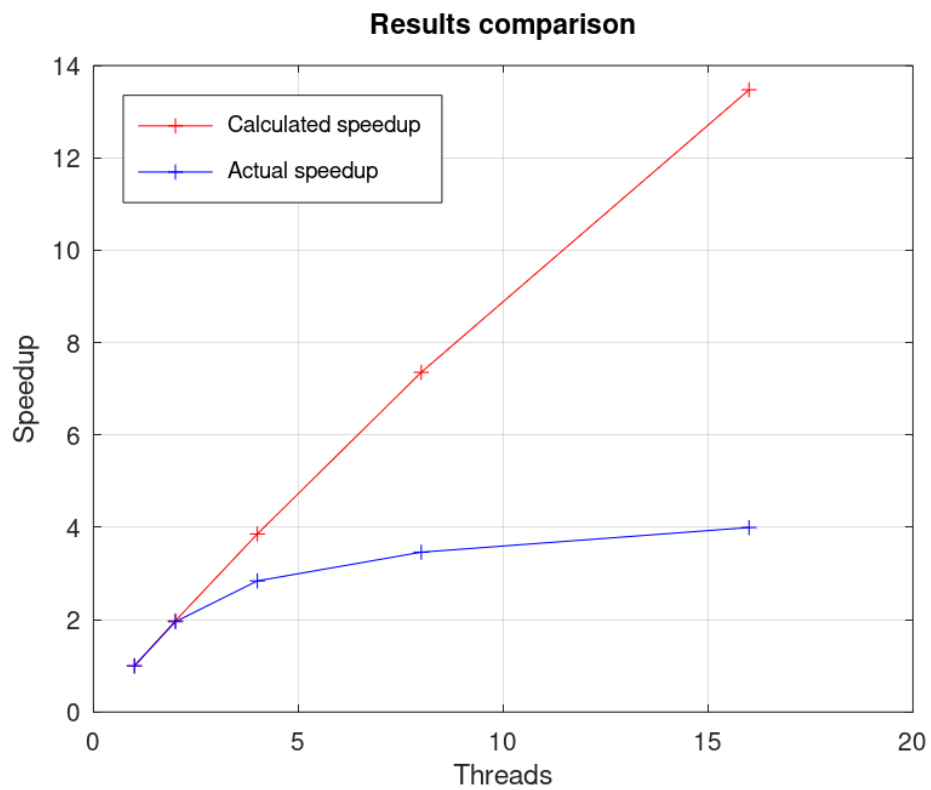


Figure 6.8: Plot comparing theoretical speedup calculated form Amdahl's Law to actual speedup obtained during testing.

Chapter 7

Conclusions

7.1 Final project's outcome

The final outcome of the project is the application that satisfied functional and nonfunctional requirements, as it was presented in table 6.1. Application allows to process images on the server. Client can send pair of files and manage account. Client can be run multiple times to request processing of the files.

Application can be run on multiple platforms and multiple architectures. It was tested on MacOS, Windows 10 and multiple Linux distributions. Additionally it was tested on AArch64 and AMD64 processors.

7.2 Possible future development

One way of improving performance of the project, is to change current database of choice to Redis Cache for storing BLOBs. This could vastly improve their search, on expense of much higher CPU and RAM requirements.

Another great option to improve the performance of feature detection, is to save the matrix of keypoints to the corresponding record in the database. Instead of computing them each time, they can be stored, and retrieved when needed. This will reduce performance of processing new files, but processing files that are already stored inside the database can vastly reduce time, to just the brute-force feature matching.

7.3 Encountered difficulties and problems

The biggest problem encountered at the beginning of development process was missing *DrawMatches* function form GoCV bindings of OpenCV. At that point, I was starting learning Go, and decided to implement this function myself, creating fork of GoCV and eventually merging it to the upstream.

Bibliography

- [1] Michael Banks. *On the Way to the Web: The Secret History of the Internet and its Founders*. Apress, 2012.
- [2] Jacquelyn Bulao. How much data is created every day in 2020? <https://techjury.net/blog/how-much-data-is-created-every-day>, September 2020.
- [3] Brendan Burns. *Designing Distributed Systems: Patterns and Paradigms for Scalable, Reliable Services*. O'Reilly Media, Inc., 2018.
- [4] Larry Dignan. Top cloud providers in 2020: Aws, microsoft azure, and google cloud, hybrid, saas players. <https://www.zdnet.com/article/the-top-cloud-providers-of-2020-aws-microsoft-azure-google-cloud-hybrid-saas/>, October 2020.
- [5] Docker Docs. What is a container? <https://www.docker.com/resources/what-container>.
- [6] PostgreSQL Documentation. Installation. <https://www.postgresql.org/docs/9.3/tutorial-install.html>.
- [7] PostgreSQL Documentation. Supported platforms. <https://www.postgresql.org/docs/9.5/supported-platforms.html>.
- [8] gRPC Docs. Why grpc? <https://grpc.io/>.
- [9] James Hamilton. Aws graviton processor. <https://www.youtube.com/watch?v=LNqRvP6Xvrw>, June 2020.

- [10] Mark D. Hill. Amdahl's law in the multicore era. *Computer*, 41(7), 2008.
- [11] Unmesh Joshi. Patterns of distributed systems - heartbeat. <https://martinfowler.com/articles/patterns-of-distributed-systems/>, August 2020.
- [12] Xiaolin Li. Distributed image processing on a network of workstations. *International Journal of Computers and Applications*, 25(2), 2003.
- [13] David G. Lowe. Distinctive image features from scale-invariant keypoints. Technical report, Computer Science Department, University of British Columbia, Vancouver, B.C., Canada, January 2004.
- [14] University of California. Berkeley open infrastructure for network computing. <https://boinc.berkeley.edu/>.
- [15] Stefano Tempesta. Design patterns for distributed systems. <https://www.codemag.com/Article/1909071/Design-Patterns-for-Distributed-Systems>.
- [16] Chang-Sung Jeong Yoon-Ki Kim, Yongsung Kim. Ride: real-time massive image processing platform on distributed environment. *EURASIP Journal on Image and Video Processing*, 2018.

Appendices

List of abbreviations and symbols

CCTV Closed-circuit television, video surveillance

CLI Command Line Interface

CPU Central Processing Unit

DISC International Symposium on Distributed Computing

DLT Distributed Ledger Technology

gRPC gRPC Remote Procedure Call Framework

HDD Hard Disk Drive

NOWs Network Of Workstations

PaaS Platform as a service

PODC Symposium on Principles of Distributed Computing

PVE Proxmox Virtual Environment

RIDE Real-time massive image processing platform on distributed environment

RPC Remote Procedure Calls

SSD Solid State Drive

WSL(2) Windows Subsystem for Linux (2)

Listings

Best Matches

```
1 func BestMatches(query string, train string) gocv.Mat {
2
3     img1 := gocv.IMRead(query, gocv.IMReadGrayScale)
4     defer img1.Close()
5     img2 := gocv.IMRead(train, gocv.IMReadGrayScale)
6     defer img2.Close()
7
8     sift := gocv.NewSIFT()
9     defer sift.Close()
10
11     kp1, des1 := sift.DetectAndCompute(img1, gocv.NewMat
12         ())
13
14     kp2, des2 := sift.DetectAndCompute(img2, gocv.NewMat
15         ())
16
17     bf := gocv.NewBFMatcher()
18     matches := bf.KnnMatch(des1, des2, 2)
19
20     var good []gocv.DMatch
21     for _, m := range matches {
22         if len(m) > 1 {
23             if m[0].Distance < 0.75*m[1].Distance {
```



```
21         good = append(good, m[0])
22     }
23 }
24 }
25
26     c1 := color.RGBA{
27         R: 0,
28         G: 255,
29         B: 0,
30         A: 0,
31     }
32
33     c2 := color.RGBA{
34         R: 255,
35         G: 0,
36         B: 0,
37         A: 0,
38     }
39
40     mask := make([]byte, 0)
41
42     out := gocv.NewMat()
43
44     gocv.DrawMatches(img1, kp1, img2, kp2, good, &out, c1
45         , c2, mask, gocv.DrawDefault)
46
47     return out
48 }
```

Tables

On the following pages large tables are presented.

Table 1: Performance results of system processing 80 files.

Threads	Elapsed time	User time	System time
1	4:13:32	1559.34	626.10
2	2:10:51	630.60	277.50
2	2:08:19	621.15	271.32
4	1:30:30	364.14	163.08
4	1:29:00	360.27	157.54
4	1:28:57	355.54	159.76
4	1:28:40	357.22	160.11
8	1:14:16	186.82	83.24
8	1:13:56	192.65	81.42
8	1:14:10	190.09	82.40
8	1:11:58	174.00	78.76
8	1:10:43	174.94	75.96
8	1:14:24	190.82	84.01
8	1:13:16	184.80	81.01
8	1:13:16	179.15	79.43
16	1:05:13	97.90	43.74
16	1:04:49	97.61	43.72
16	1:04:32	97.84	45.42
16	1:04:44	100.72	45.34
16	1:05:48	102.66	45.43
16	1:03:52	95.63	43.91
16	1:02:57	95.16	41.63
16	0:59:11	86.27	42.82
16	0:54:13	78.91	37.18
16	1:05:50	102.16	47.34
16	1:05:28	98.72	46.74
16	1:04:55	96.56	45.40
16	1:04:32	90.86	43.90
16	1:04:36	99.89	44.81
16	0:59:31	89.25	42.61
16	1:05:03	96.91	44.42

Diagrams

On the following pages better quality diagrams are presented.

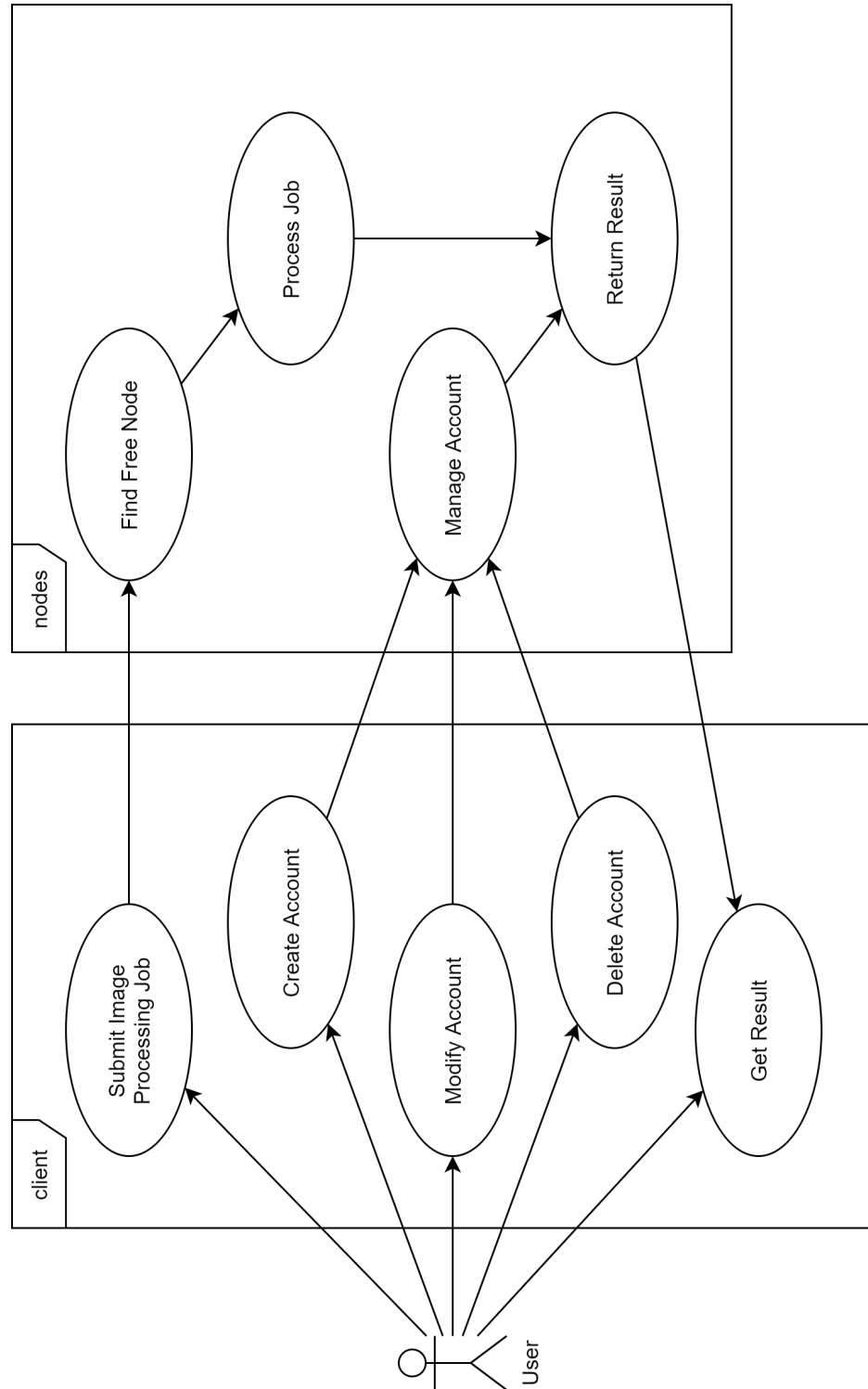


Figure 1: High quality use case diagram.

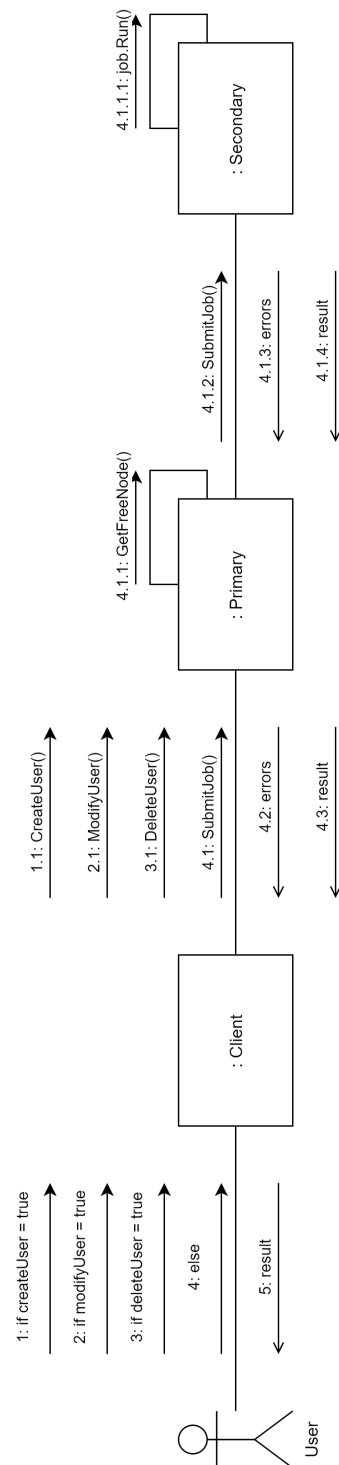


Figure 2: High quality communication diagram.

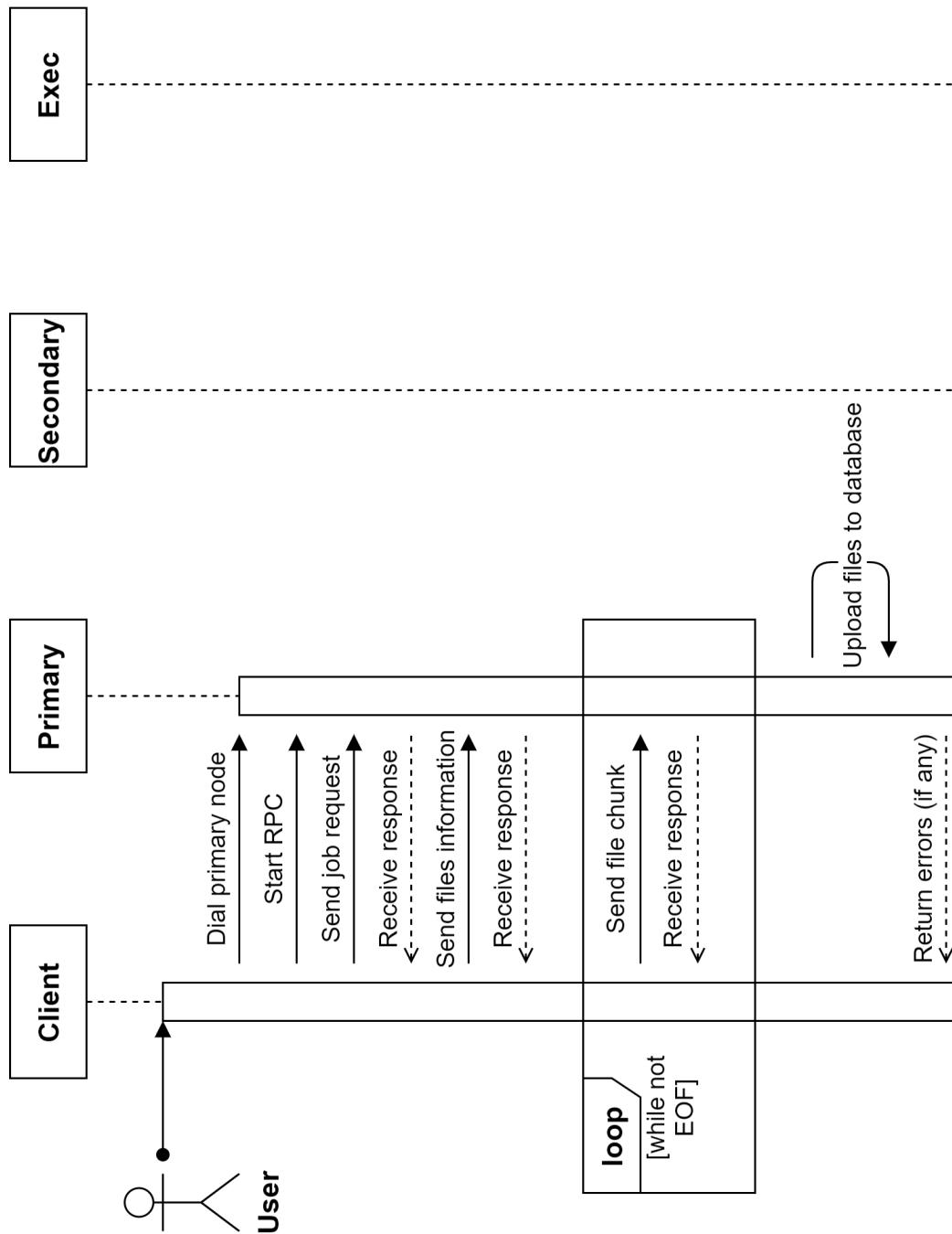


Figure 3: High quality sequence diagram of job RPC - part 1.

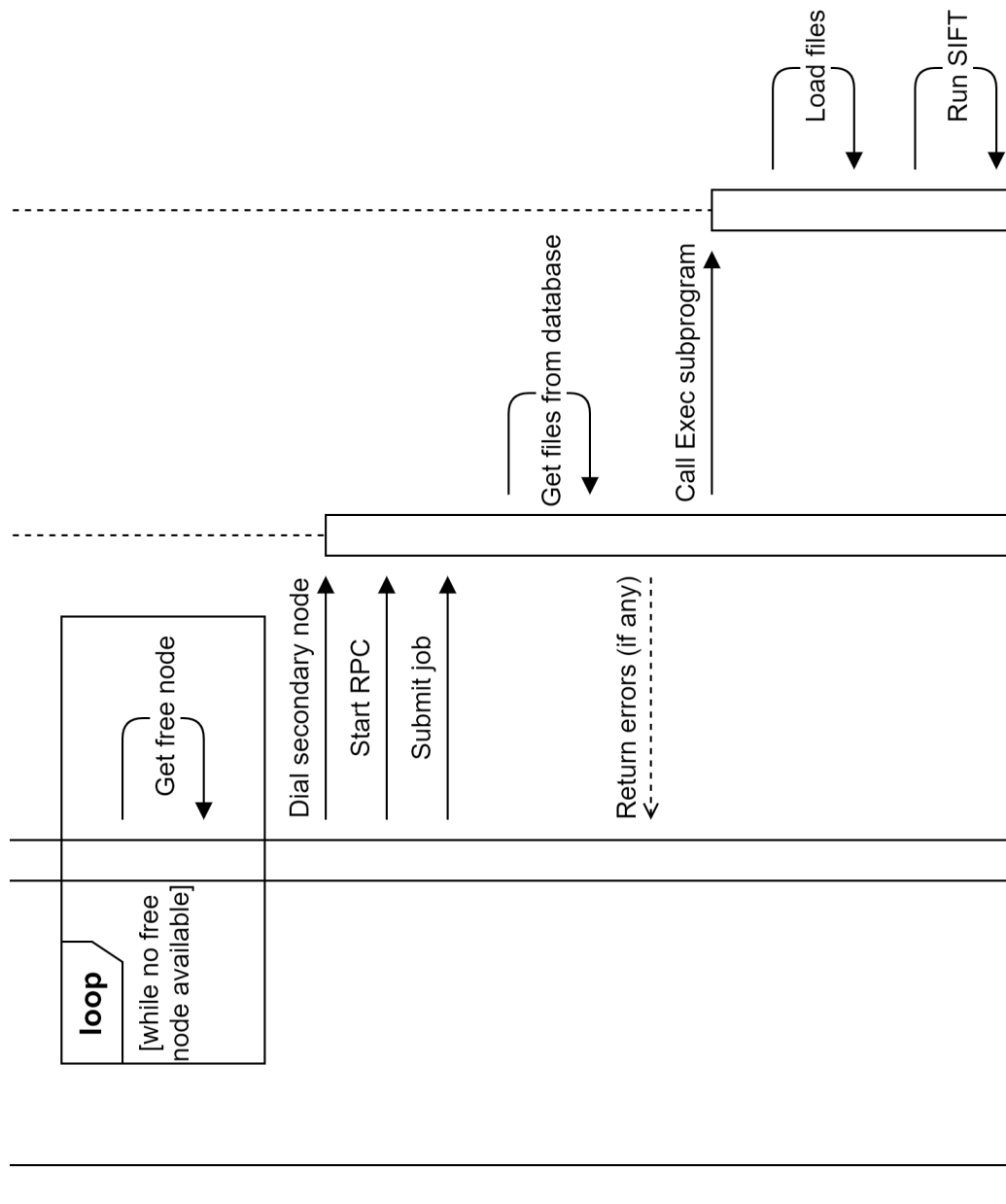


Figure 4: High quality sequence diagram of job RPC - part 2.

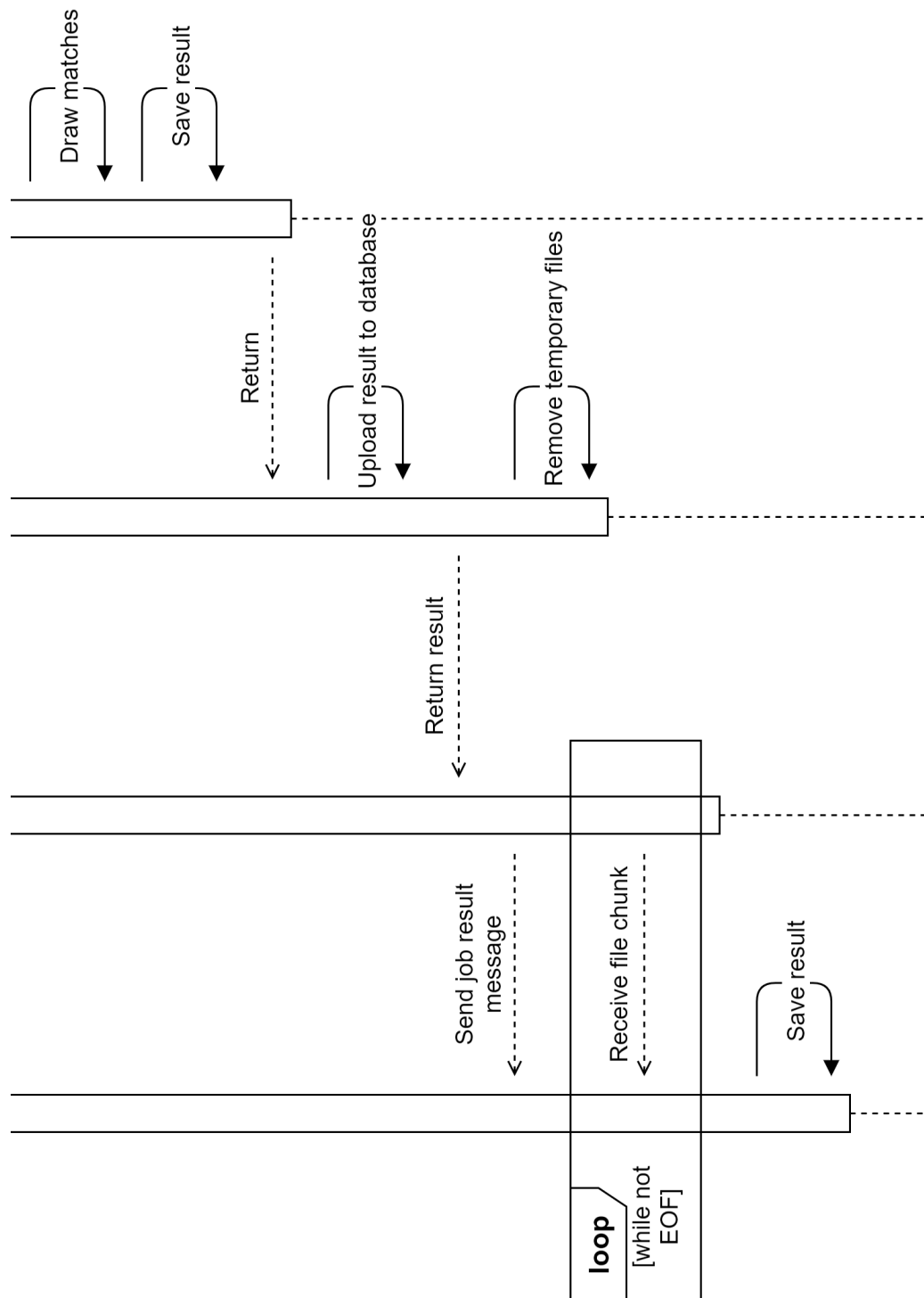


Figure 5: High quality sequence diagram of job RPC - part 3.

Contents of attached CD

The thesis is accompanied by a CD containing:

- thesis (L^AT_EX source files and final pdf file),
- source code of the application,
- test data.

List of Figures

3.1	Use case diagram.	7
4.1	Communication diagram of Project.	11
4.2	Sharded service diagram based on diagram from <i>Designing Distributed Systems</i>	12
4.3	Sequence diagram of job RPC	14
4.4	Sequence diagram of user management RPCs.	16
4.5	Entity Relation Diagram of database used in project.	17
5.1	Function that is run as subroutine for updating the timestamp in the database.	24
5.2	Part of computation subprogram responsible for computation. . . .	26
5.3	Deployment diagram of application.	27
5.4	The docker-compose deployment description file example.	28
5.5	Example of Client configuration file.	31
5.6	Example of Manager node configuration file.	31
5.7	Example of Worker node configuration file.	32
5.8	Example of database connection configuration file.	32
6.1	Result of image processing on OpenCV's exemplary images, <i>box</i> and <i>box_in_scene</i> using default exec program.	36
6.2	Plot showing timing decrease as a result of adding Workers, compared to unobtainable ideal solution.	39
6.3	Plot showing speedup in processing of 80 files.	40
6.4	Plot showing timing increase, compared to unobtainable ideal solution.	41

6.5	Plot showing speedup in processing of 4 big files.	42
6.6	Plot showing timing decrease, compared to unobtainable ideal solution.	43
6.7	Plot showing speedup in processing of 4 big files after changing the deployment method.	44
6.8	Plot comparing theoretical speedup calculated form Amdahl's Law to actual speedup obtained during testing.	46
1	High quality use case diagram.	XVI
2	High quality communication diagram.	XVII
3	High quality sequence diagram of job RPC - part 1.	XVIII
4	High quality sequence diagram of job RPC - part 2.	XIX
5	High quality sequence diagram of job RPC - part 3.	XX

List of Tables

6.1	Fulfilled Requirements	35
6.2	Performance results of system processing 4 files using cache mechanism from beginning.	37
6.3	Performance results of system processing 4 files using cache mechanism from beginning and with changed deployment method. . . .	38
6.4	Comparison of the number of threads and average elapsed time during processing of 80 files.	38
6.5	Comparison of the number of threads and average elapsed time during processing of 4 files.	40
6.6	Comparison of the number of threads and average elapsed time during processing of 4 files after changing the deployment method. . . .	42
6.7	Comparison of the theoretical and actual speedup of processing 80 images	45
1	Performance results of system processing 80 files.	XIV